

HelixSkills

HelixSkills

Регулируемая конституционно система навыков для агентов CLI AI.

Краткое описание

HelixSkills — это система навыков для агентов CLI AI, наследующая Helix Constitution в качестве субмодуля, что гарантирует безусловное применение всех универсальных правил управления. Она объединяет устанавливаемые навыки агентов, серверы инструментов MCP, плагины Claude Code и многоразовые движки в единый регистрируемый и документированный каталог.

Краткая аннотация

HelixSkills — система навыков для агентов CLI AI. Она интегрирует Helix Constitution как субмодуль, обеспечивая действие всех универсальных правил, а затем предоставляет регистрируемые навыки (префиксы действий, валидатор медиа, мультитрек, синхронизация сессий, жизненный цикл рабочих элементов и другие), два сервера инструментов MCP, два плагина Claude Code и многоразовые движки.

Подробное описание

HelixSkills (репозиторий `skills`, лицензия Apache-2.0) — это система навыков для агентов CLI AI, построенная на принципиально ином подходе: сначала управление, затем возможности. Она наследует Helix Constitution как субмодуль `constitution/`, поэтому все универсальные правила из `constitution/CLAUDE.md` и `constitution/Constitution.md` применяются безоговорочно — не как соглашение, которое агент может соблюдать по желанию, а как набор правил,

физически встроенных в структуру проекта. Агент, использующий HelixSkills, не может отказаться от конституции: правила следуют за кодом.

В то время как большинство «фреймворков навыков» оперируют абстракциями, HelixSkills предоставляет конкретный, регистрируемый каталог, который можно установить напрямую. Семь конституционных навыков устанавливаются через `register.sh`: `action-prefix-system`, `media-validator`, `multitrack`, `reporting-workable-items`, `scheduled-work-queue`, `session-sync` и `workable-item-lifecycle` — спектр от среднего до продвинутого уровня, охватывающий всё: от строгого именования действий до валидации медиа и полного жизненного цикла рабочей единицы. Дополнительные черновые навыки (обзор Android, язык Java/Kotlin, ОС Linux) уже индексированы и подготовлены к активации. Два сервера инструментов MCP (`media-validator` и `scheduled-work`) предоставляют эти навыки агентам через Model Context Protocol, а два плагина Claude Code (`helix` и `scheduled-work`) внедряют те же возможности непосредственно в среду выполнения агента — один набор навыков, доступный агентам через любой интерфейс.

В основе каталога лежат четыре многоразовых движка первого уровня — `continuum` (реализован), а также `session_orchestrator`, `token_optimizer` и `clickup_sync` (на стадии проектирования) — общая инфраструктура, исключая повторное изобретение базовых механизмов. Один лишь `token_optimizer` содержит явный граф зависимостей, охватывающий пакеты экосистемы `vasic-digital` (`TOON`, `Embeddings`, `VectorDB`, `Normalize`, `conversation`) и `LLMProvider` из `HelixDevelopment`, что делает межрепозиторные связи прозрачными для аудита, а не неявными. Всю систему сопровождает строгая документация: каталог навыков, автоматически генерируемый индекс графа навыков, детальные страницы по каждому репозиторию и откровенный реестр «Пробелы и риски», где перечислено то, что ещё не реализовано. Вся система дублируется на GitHub, GitLab, GitFlic и GitVerse для обеспечения отказоустойчивости и доступа в разных регионах.

Почему мы его создали

Агентам CLI и AI нужны возможности, которые были бы согласованными, управляемыми и многоразовыми — а не разрозненные скрипты, каждый из которых заново изобретает правила. HelixSkills был создан, чтобы предоставить агентам готовый, регистрируемый набор навыков, привязанный к

общей конституции, благодаря чему поведение остаётся единообразным и поддаётся аудиту в каждом агенте и проекте, где он используется.

Почему это меняет правила игры

Он делает возможности агентов переносимыми и соответствующими правилам *по определению*, а не за счёт дисциплины. Каждый навык — это управляемый, версионированный и устанавливаемый модуль, подкреплённый подмодулем конституции, поэтому в момент регистрации навыка агент автоматически наследует эталонный свод правил без возможности отклонений. Это открывает то, что раньше было нереализуемо: перенос возможностей из одного агента или проекта в другой с уверенностью, что они придут уже привязанными к той же системе управления, доступными через стандартные интерфейсы (серверы MCP и плагины Claude Code), а не через нагромождение самописных скриптов, каждый из которых заново изобретает правила.

Что здесь нового

- **Constitution как подмодуль:** универсальные правила управления наследуются, а не копируются — подключаются в дерево проекта, так что каждый агент, использующий их, привязан к одному и тому же эталонному своду правил, а обновления поступают из единого источника истины, а не из десятка устаревших копий.
- Навыки поставляются как саморегистрируемые модули (`register.sh`) и интегрируются в автоматически генерируемый индекс графа навыков, благодаря чему каталог остаётся актуальным и никогда не расходится с тем, что реально установлено.
- **Мультиплатформенная доступность:** один и тот же набор навыков доступен агентам через серверы инструментов MCP и плагины Claude Code — напиши один раз, работай в любой среде исполнения, которую использует агент.
- Многоразовые движки первого уровня (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`), общие для всей экосистемы, каждый из которых содержит явные и поддающиеся аудиту декларации межрепозиторных зависимостей вместо скрытых связей.

Главные технические вызовы и как мы их преодолели

- **Сохранение единообразия поведения агентов и соответствия правилам при большом количестве навыков и агентов** — повторная реализация системы управления для каждого навыка гарантирует расхождение со временем. Решение: подключение Helix Constitution как подмодуля, чтобы правила из `constitution/CLAUDE.md` и `constitution/Constitution.md` применялись безусловно и обновлялись из единого источника, а не копировались и оставались без внимания.
- **Обеспечение устанавливаемости и обнаруживаемости растущего набора навыков** — каталог бесполезен, если никто не может найти или установить то, что в нём есть. Решение: регистрация каждого навыка через `register.sh`, которая подключает его при установке, плюс автоматически генерируемый индекс графа навыков INDEX и документация по каждому репозиторию, чтобы поиск всегда отражал реальное состояние.
- **Работа с агентами, использующими разные среды исполнения** — одну и ту же возможность не нужно переписывать под каждый хост. Решение: упаковка единого набора навыков как в определения серверов инструментов MCP (в `constitution/mcp/`), так и в плагины Claude Code (в `constitution/plugins/`), чтобы одна реализация была доступна на всех платформах.

Технологический стек

- **Shell (основной язык)** — выбран, поскольку инструменты установки и регистрации должны работать везде, где существует агент, без предварительной загрузки рантайма; на нём написаны `register.sh` и `install_upstreams`, что делает процесс подключения независимым от зависимостей и переносимым.
- **Git-сабмодули** — выбраны для наследования правил без дублирования: Helix Constitution подключён в директорию `constitution/` как живая ссылка, благодаря чему обновления распространяются через один указатель, а не копируются и забываются.
- **Model Context Protocol (MCP)** — выбран в качестве стандартного, независимого от рантайма интерфейса для агентов; два MCP-сервера (`media-validator` и `scheduled-work`) определены в `constitution/mcp/` и предоставляют навыки в виде вызываемых инструментов.
- **Плагины Claude Code** — выбраны для встраивания навыков непосредственно в рантайм агента без дополнительных связующих элементов; два плагина (`helix` и

scheduled-work) поставляются в constitution/plugins/, дублируя поверхность MCP для другого хоста.

- **Многоразовые движки (continuum, token_optimizer, session_orchestrator, clickup_sync)** — выбраны для вынесения общей логики за пределы отдельных навыков с целью повторного использования в разных проектах; например, token_optimizer интегрирован с пакетами vasic-digital (TOON, Embeddings, VectorDB, Normalize, conversation) и HelixDevelopment LLMProvider через объявленные зависимости, а не дублированный код.
- **Зеркалирование Git на нескольких хостах (GitHub, GitLab, GitFlic, GitVerse)** — выбрано, чтобы отключение одного хоста или региональная блокировка не могли перекрыть доступ; один и тот же репозиторий поддерживается в актуальном состоянии на четырёх платформах для обеспечения отказоустойчивости и доступности.

Статус и замечания о прозрачности

- **Статус: бета-версия.** Семь конституционных навыков, два MCP-сервера и два плагина уже выпущены; черновые навыки проиндексированы и ожидают активации, а три из четырёх движков первого уровня (session_orchestrator, token_optimizer, clickup_sync) всё ещё находятся на стадии проектирования.
- В README проект упоминается как helix_skills; канонический путь на GitHub — HelixDevelopment/skills. Количество обнаруженных находок в README указано на основе самоотчёта.

Приоритетный уровень: Helix-первичный.